

Anomaly Detection in Application Logs with Attention Mechanism-Enhanced LSTM

Adetiya Bagus Nusantara

Department of Informatics

Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia

6025231077@student.its.ac.id

Hudan Studiawan

Department of Informatics

Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia

hudan@its.ac.id

Royyana Muslim Ijtihadie

Department of Informatics

Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia

roy@its.ac.id

Abstract—As digital technology continues to spread across various sectors, the need for robust cybersecurity becomes increasingly critical. One way to ensure security is by identifying potential threats and anomalies through application log analysis. Traditional methods that rely only on rule-based detection are often insufficient to adapt to new attack patterns. This research addresses these limitations by integrating rule-based detection with Sigma rules into the ELK (Elasticsearch, Logstash, Kibana) platform to generate a structured dataset. Logs are collected from monitored applications and categorized into normal, system anomalies, or user anomalies. A modified long short-term memory (Lstm) model with an attention mechanism is trained on the labeled dataset to improve anomaly detection. Experimental results show that the LSTM model with the attention mechanism achieves the highest performance, with an accuracy of 98.52%, precision of 98.66%, recall of 98.52%, and an F1-score of 98.55%, outperforming RNN, GRU, and standard LSTM models in anomaly detection.

Index Terms—anomaly detection, deep learning, rule-based, cybersecurity, application logs, long short term memory

I. INTRODUCTION

The growing adoption of digital technology in various sectors, including business, education, and government, continues to expand. The use of digital technology offers many benefits, such as simplifying tasks and increasing efficiency across various sectors. However, this increasing reliance on digital systems also increases the risk of cyber threats, such as attacks, data breaches, and unauthorized access, which can compromise system security and integrity [1]. As digital infrastructures become more interconnected, the potential attack surface also expands, providing more opportunities for cybercriminals to exploit vulnerabilities [2]. The increasing complexity of these attacks requires more advanced and proactive security strategies [2]. Several approaches, such as anomaly detection, have been widely used to enhance cybersecurity awareness.

The complexity of attack types has increased over time, making detection and prevention more challenging [3]. Traditional security measures are often no longer sufficient to handle these evolving threats. As a result, organizations need to adopt more advanced approaches. An effective method is to analyze application logs to identify unusual patterns or suspicious activities that may indicate potential threats. By detecting unusual or suspicious activities, systems can identify potential threats early and prevent more significant damage [4].

In cybersecurity, the rule-based anomaly detection approach is commonly used to identify suspicious or abnormal activities. This method works by applying a predefined set of rules or logic to detect anomalies [5]. These rules can be simple, such as detecting when the number of failed login attempts exceeds a certain threshold. More complex rules may involve identifying unusual activity patterns at specific times. For example, if a system detects repeated automated requests resembling SQL injection attempts from an agent such as SQLMap, it can flag the activity as a potential threat and take preventive measures.

The advantage of the rule-based anomaly detection method is its simplicity, which allows for quick and accurate results when the rules are clearly defined [6]. However, this approach has limitations in flexibility, especially in detecting new (zero-day) attacks or anomalous behaviors that have not been previously defined [7]. Rules must be manually updated to address evolving threats. This process requires expert knowledge to keep the rules relevant. In addition, the system must be continuously updated to effectively counter new threats.

In this paper, a performance comparison of different deep learning models, including RNN, GRU, LSTM, and LSTM with an attention mechanism, is conducted to detect anomalies in application logs. The increasing complexity of cyber threats has made traditional rule-based anomaly detection less effective, as predefined rules struggle to identify unknown or evolving attack patterns. This study aims to evaluate the effectiveness of deep learning in improving anomaly detection accuracy by leveraging structured datasets generated through rule-based method.

Contributions. The contribution of this paper is the proposed use of LSTM with attention to improve threat detection in cybersecurity. This method is expected to improve the ability to detect anomalies while addressing the limitations of traditional rule-based techniques. The impact of this approach is observed in its ability to increase the detection accuracy of anomaly logs and provide more adaptive threat identification in dynamic digital environments.

This paper is structured into five sections. Section I provides an introduction to the research, outlining the background, and objectives of the research. Section II discusses related work on anomaly detection in application logs using rule-

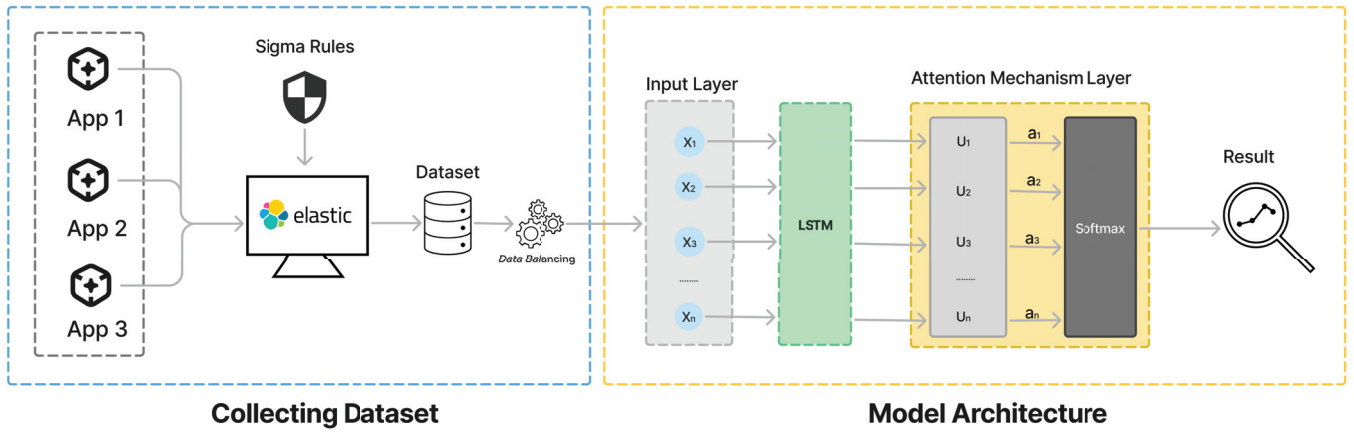


Fig. 1. Illustration of the proposed method.

based methods and deep learning. Section III provides a detailed explanation of the proposed approach, including the integration of rule-based techniques for dataset generation and the deep learning model for anomaly detection. Section IV presents the performance evaluation of the proposed method. Finally, Section V concludes the research.

II. RELATED WORK

Previous studies have explored various approaches, including rule-based methods and deep learning techniques, to automate the detection of anomalies. Hermawan et al. [8] developed a threat-hunting platform using the ELK stack and Sigma rules for attack detection. The platform successfully identified attack patterns from penetration testing scenarios. However, the researchers proposed incorporating machine learning for real-time attack detection to tackle the growing complexity of cyber threats for future research.

Deep learning techniques have been widely applied to anomaly detection in logs. Zhao et al. [9] proposed a bidirectional LSTM-based anomaly detection model trained on over one billion log entries from LANL's internal network. The proposed model outperformed the one-class SVM, Gaussian mixture models (GMM), and principal component analysis (PCA) in terms of accuracy and efficiency, as measured by AUC.

Several studies have enhanced LSTM-based models with attention mechanisms for improved anomaly detection. Pranolo et al. [10] incorporated attention into LSTM, coupled with Grid Search and PSO, to predict energy demand. The proposed model reduced MAPE from 0.39 to 0.37 and increased R^2 from 0.901 to 0.908. Furthermore, Zou et al. [11] proposed an attention-enhanced LSTM model for latency prediction in Mobile Edge Computing (MEC). The proposed approach achieved an accuracy of 0.83, outperforming CNN (0.80) and standard LSTM (0.72).

Kang et al. [12] applied an attention-based LSTM model to predict the position of the shield machine in tunnel construction. The model improved the average R^2 from 0.625 to

0.736 and reduced RMSE from 3.31 to 2.24, demonstrating its effectiveness in enhancing prediction accuracy. These studies highlight the effectiveness of integrating attention mechanisms with LSTM models to improve anomaly detection in log data. Therefore, it motivates us to further investigate hybrid approaches that combine rule-based and deep learning techniques.

III. PROPOSED METHOD

This chapter outlines the proposed method in this research. It focuses on two main aspects: dataset collection using ELK and the application of deep learning for anomaly detection. The dataset is gathered using the ELK stack (Elasticsearch, Logstash, and Kibana) as the platform for managing and analyzing application logs. Through ELK, logs from various sources are collected and stored in a structured manner to perform the identification of anomaly patterns. After the dataset is obtained, the next step is building the deep learning model. This chapter also explains the implementation steps and the model architecture in model training. The overview of the proposed method is illustrated in Fig. 1.

A. Collecting Datasets

The process of obtaining the dataset begins with the implementation of Sigma rules, which are globally recognized rule-based detection mechanisms designed to identify anomalies and security threats within log data. Since Sigma rules are written in a generic format, these rules must be converted into a format compatible with ELK. The detailed process of applying these rules to identify anomalies is outlined in Algorithm 1.

After the rules are integrated into ELK, Application Performance Monitoring (APM) agents are installed on each application server. These agents collect real-time data related to application performance, error tracking, and request tracing. The gathered data is then forwarded to the ELK stack.

B. Data Balancing

The collected dataset is categorized into three classes: normal logs, user anomaly logs, and system anomaly logs. Normal logs represent routine system activities without any signs of suspicious behavior. User anomaly logs capture irregular or potentially malicious activities performed by users, such as repeated failed login attempts or unauthorized access attempts. Meanwhile, system anomaly logs include unexpected system behaviors, such as unusual error patterns, service crashes, or network anomalies. This categorization provides a structured dataset that allows the model to differentiate between different types of anomalies.

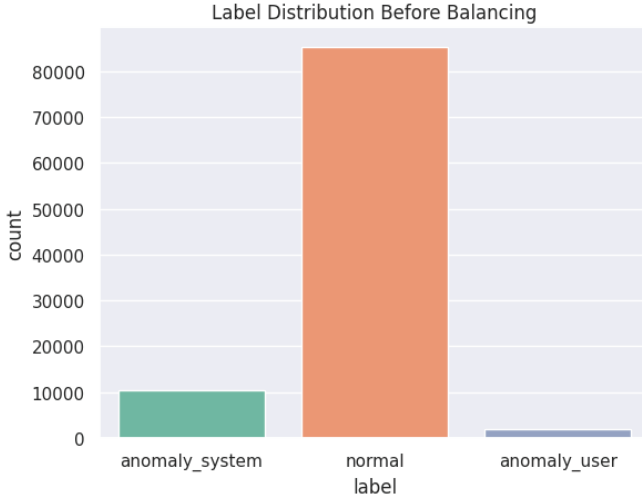


Fig. 2. Label distribution before balancing.

However, the initial dataset is imbalanced, with normal logs significantly outnumbering anomaly logs as shown in Fig. 2. The dataset consists of 85,264 normal logs, 10,455 system anomaly logs, and 1,867 user anomaly logs. This imbalance can negatively impact the performance of deep learning models [13]. It leads to biased predictions where the model favors the majority class and struggles to detect rare anomalies [13]. To address this issue, undersampling is applied to the normal log category. The undersampling process is carefully designed to retain essential variations in normal log data while avoiding excessive information loss [14]. Random sampling is employed to ensure that different types of normal

system activities remain represented. Meanwhile, all available user and system anomaly logs are preserved to maintain the rarity and significance.

Mathematically, the class imbalance ratio R can be expressed as:

$$R = \frac{N_{\text{majority}}}{N_{\text{minority}}} \quad (1)$$

where N_{majority} represents the number of instances in the majority class (normal logs), and N_{minority} represents the number of instances in the minority class (anomalous logs). High values of R indicate a severe imbalance.

To perform undersampling, the number of majority class samples N'_{normal} is reduced according to:

$$N'_{\text{normal}} = \alpha N_{\text{minority}}, \quad \alpha > 1 \quad (2)$$

where α is a scaling factor that ensures balance while retaining sufficient data diversity. The goal is to achieve an optimal class distribution where:

$$N'_{\text{normal}} \approx N_{\text{anomaly_user}} + N_{\text{anomaly_system}} \quad (3)$$

After applying undersampling, the number of data points in each category becomes more balanced, with a total of 1,494 data for each category as shown in Fig. 3.

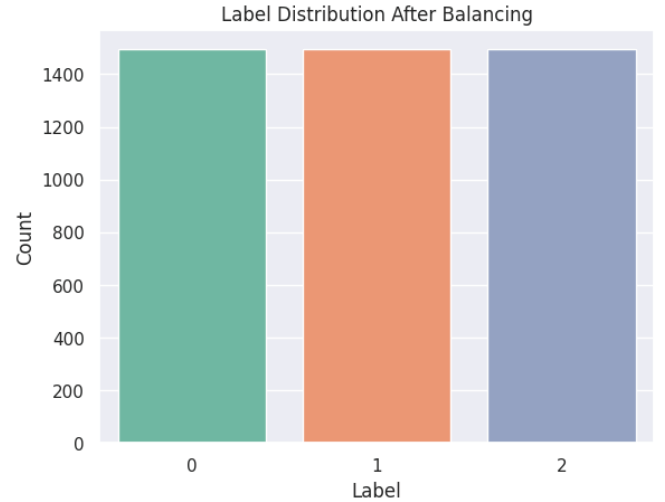


Fig. 3. Label distribution after balancing.

Algorithm 1 Detecting anomalies using Sigma rules

```

logs ← read system logs
rules ← read Sigma rules
for each log in logs do
  for each rule in rules do
    if rule.keyword is found in log then
      mark log as an anomaly
    end if
  end for
end for

```

With the balanced dataset, the deep learning model can learn more effectively without being overwhelmed by the dominance of the majority class. [14].

C. Text Preprocessing

In this research, text preprocessing is used to convert raw textual data into a structured format suitable for deep learning models. This process involves tokenization, sequence padding, and word embedding using GloVe (Global Vectors for Word Representation) [15].

The first step in preprocessing is tokenization, where text is broken down into smaller units called tokens. In this research, a Keras Tokenizer is used to convert words into numerical sequences. The tokenizer is trained exclusively on the training dataset to prevent data leakage [15]. Additionally, an out-of-vocabulary (OOV) token is defined to handle unseen words during testing [15]. Once tokenization is completed, the text is transformed into sequences of integers, with each number representing the index of a word in the vocabulary.

Mathematically, given a sentence S consisting of n words:

$$S = \{w_1, w_2, \dots, w_n\} \quad (4)$$

the tokenizer maps each word w_i to a unique integer index t_i :

$$T = \{t_1, t_2, \dots, t_n\}, \quad t_i = \text{Tokenizer}(w_i) \quad (5)$$

where T represents the tokenized sequence.

Since deep learning models require inputs of uniform length, padding is applied to ensure that all sequences have the same length, either by truncating longer sequences or padding shorter ones with zeros [15]. If the maximum sequence length is defined as L , then:

$$T' = \begin{cases} (t_1, t_2, \dots, t_L) & \text{if } n > L \text{ (truncate)} \\ (t_1, t_2, \dots, t_n, 0, \dots, 0) & \text{if } n < L \text{ (pad with zeros)} \end{cases}$$

where T' represents the fixed-length sequence.

After tokenization, word embeddings are generated using GloVe, a pre-trained word vector representation technique. The embeddings are obtained from a pre-trained word vector model, where each word is represented as a 100-dimensional vector based on word co-occurrence patterns in a large text corpus [15].

Given a vocabulary of size V and an embedding dimension d , the word embedding matrix E is defined as:

$$E \in \mathbb{R}^{V \times d} \quad (6)$$

where each word w_i is represented as a vector:

$$v_i = E[w_i], \quad v_i \in \mathbb{R}^d \quad (7)$$

Words that do not exist in the GloVe dictionary are initialized with zero vectors:

$$v_{\text{unknown}} = \mathbf{0}_d \quad (8)$$

This preprocessing pipeline ensures that textual data is converted into numerical representations that preserve semantic meaning [15].

D. Proposed Attention-Enhanced LSTM Model

To improve anomaly detection, this research employs a long short-term memory (LSTM) model with an integrated attention mechanism. The architecture consists of multiple layers, each contributing to the effective processing of sequential text data. The model starts with an input layer obtained from the previous preprocessing step. Next, an LSTM layer with 64 hidden units is applied to capture both past and future dependencies in the sequence:

$$H = \text{LSTM}(E) \in \mathbb{R}^{L \times 2h} \quad (9)$$

where h is the number of LSTM units in each direction.

To enhance feature extraction, an attention mechanism is introduced. The attention mechanism improves feature extraction by allowing the model to focus on the most relevant time steps [11]. This mechanism assigns different importance scores to each LSTM output, ensuring that key patterns in the log data are effectively captured [11].

$$A = \text{Attention}(H, H) \in \mathbb{R}^{L \times 2h} \quad (10)$$

where A is the attention-weighted representation of the LSTM outputs. Then, to further refine the extracted features, Global Average Pooling is applied:

$$V = \frac{1}{L} \sum_{i=1}^L A_i \quad (11)$$

where V is the final feature vector extracted from the attention mechanism. The pooled attention features pass through a fully connected dense layer with ReLU activation:

$$Z = \text{ReLU}(WV + b) \quad (12)$$

where W and b are the weight and bias parameters.

To improve generalization and prevent overfitting, a dropout layer with a probability of 0.5 is applied. Finally, the model performs classification using a softmax output layer:

$$\hat{Y} = \text{softmax}(W_o Z + b_o) \quad (13)$$

where $\hat{Y}$ represents the predicted class probabilities. This well-organized process allows the model to accurately identify key features while enhancing its reliability and strengthening its ability to detect anomalies.

E. Evaluation

To evaluate the performance of the anomaly detection model, several metrics are utilized, including Precision, Recall, F1-score, and Accuracy. These metrics offer a thorough assessment of the model's ability to differentiate between normal and anomalous instances.

1) *Precision*: Precision measures the ratio of correctly identified positive cases to the total predicted positive cases. It reflects the model's ability to minimize false positives by indicating how many of the detected anomalies are actually anomalous. [16]. Precision is calculated as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (14)$$

where:

- *TP* (True Positives): The count of anomalies that were correctly identified as anomalies by the model.
- *FP* (False Positives): The count of normal instances incorrectly classified as anomalies.

A high Precision value indicates that the model has a low False Positive (FP) rate, meaning it accurately identifies anomalies without mistakenly classifying normal events as anomalies.

2) *Recall*: Recall (also known as Sensitivity) measures the proportion of actual positive cases that are correctly identified. It indicates the model's ability to detect all existing anomalies, ensuring that minimal true anomalies are missed [16]. Recall is calculated as follows:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (15)$$

where FN (False Negatives) is the number of actual anomalies incorrectly classified as normal.

A high recall value means that the model successfully detects most anomalies, minimizing false negatives [16].

3) *F1-score*: The F1-Score is the harmonic mean of Precision and Recall, offering a balanced assessment of model performance, particularly in scenarios where there is an imbalance between false positives and false negatives [16]. It is formulated as follows:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (16)$$

A high F1-score indicates a good balance between Precision and Recall, ensuring the model performs well in identifying anomalies while minimizing misclassifications [16].

4) *Accuracy*: Accuracy measures the overall effectiveness of the model by determining the proportion of correctly classified instances, including both normal and anomalous cases, relative to the total number of predictions [16]. It is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (17)$$

where TN (True Negatives) is the number of correctly predicted normal instances [16].

IV. EXPERIMENTAL RESULTS

This section presents the experimental results comparing the performance of different deep learning models, including RNN, GRU, LSTM, and LSTM with the attention mechanism, in detecting anomalies. The evaluation is based on four key

metrics, including accuracy, precision, recall, and F1-score. The models were trained and tested using the preprocessed dataset, and the results are summarized in Table I.

TABLE I
PERFORMANCE COMPARISON OF DIFFERENT MODELS

Model	Accuracy(%)	Precision(%)	Recall(%)	F1-Score(%)
RNN	92.57	95.28	92.57	93.27
GRU	80.18	90.65	80.18	83.28
LSTM	97.58	97.91	97.58	97.66
LSTM+	98.52	98.66	98.52	98.55

From Table I, LSTM model with attention mechanism achieves the best results. LSTM with attention consistently outperforms other architectures across all metrics and demonstrates its effectiveness in anomaly detection.

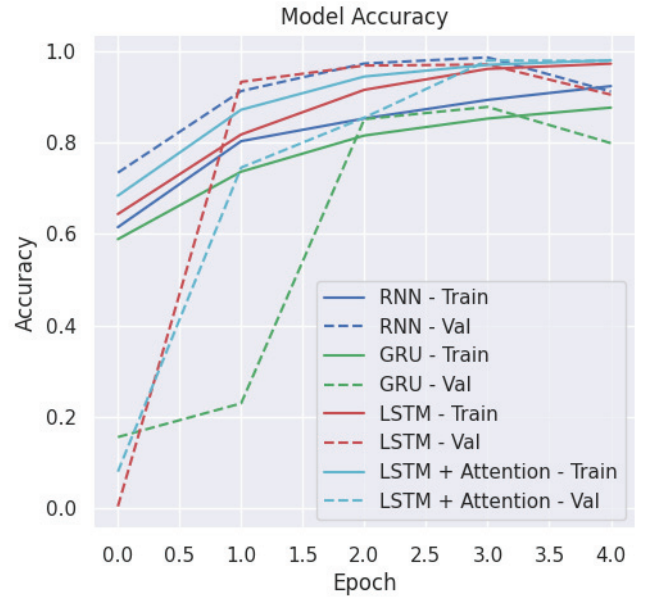


Fig. 4. Accuracy performance across training iterations.

Next, the accuracy and loss trends across different models are compared to further evaluate the performance. Fig. 4 illustrates the accuracy performance of different deep learning models across four training epochs. The x-axis indicates the number of epochs, while the y-axis denotes accuracy. The graph shows that all models experience a rapid increase in accuracy during the initial epochs. However, LSTM with attention achieves the highest accuracy, both in training and validation, and outperforms other architectures. In addition, the LSTM model demonstrates better generalization compared to GRU and RNN, as shown by the smaller gap between training and validation accuracy.

Fig. 5 illustrates the loss trends of different models across training epochs, where a decreasing loss value signifies improved model performance. The RNN model has a relatively slow decline, indicating difficulties in capturing long-term dependencies. The GRU model shows better loss reduction than

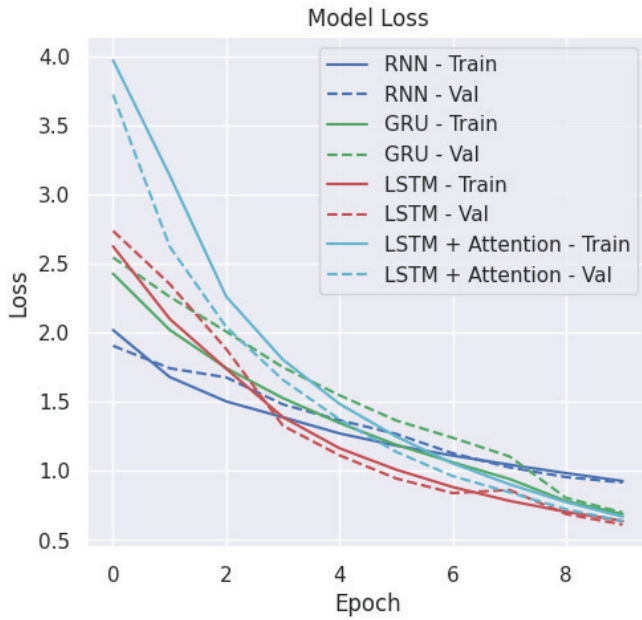


Fig. 5. Loss trends across training iterations.

RNN, but maintains higher loss values compared to the LSTM-based models. In contrast, the LSTM model demonstrates a more stable and consistent decrease in loss which reflects its ability to learn patterns effectively.

V. CONCLUSION

In conclusion, the findings indicate that the LSTM model with attention mechanism consistently outperforms other architectures in all evaluation metrics. Incorporating the attention mechanism enables the model to focus on the most relevant time steps in sequential data and improves its anomaly detection performance. This is proven by the improvement in accuracy, precision, recall, and F1-score when compared to the standard LSTM model.

These findings are further supported by the accuracy and loss trend analysis, which provide deeper insights into the model's performance during training and validation. The accuracy analysis indicates that all models experience a increase in accuracy during the initial training epochs. However, the LSTM model with the attention mechanism achieves the highest accuracy in both training and validation. Furthermore, the analysis of the loss trends shows that attention-focused LSTM exhibits a more stable and consistent reduction in loss.

For future work, exploration of Transformer-based or hybrid deep learning architectures is planned to enhance anomaly detection performance. Furthermore, implementing the proposed model in real-time scenarios and evaluating it on larger and more diverse datasets are expected to offer valuable insights into its scalability and practical applicability.

ACKNOWLEDGEMENTS

This work is financially supported by LPDP through the IN-SPIRASI Batch II under contract No. 2965/E3/AL.04/2024 between the Indonesian Ministry of Education, Culture, Research and Technology and Institut Teknologi Sepuluh Nopember.

REFERENCES

- [1] E. Soesanto, A. Romadhon, B. D. Mardika, and M. F. Setiawan, "Analisis dan peningkatan keamanan cyber: Studi kasus ancaman dan solusi dalam lingkungan digital untuk mengamankan objek vital dan file," *Sammajiva: Jurnal Penelitian Bisnis dan Manajemen*, vol. 1, pp. 172–191, 6 2023.
- [2] H. A. Thooriqoh, M. N. Azzmi, Y. A. Tofan, and A. M. Shiddiqi, "Malicious traffic detection in dns infrastructure using decision tree algorithm," *JUTI: Jurnal Ilmiah Teknologi Informasi*, pp. 45–52, 2021.
- [3] S. Das, S. Mukherjee, and S. Acharyya, "Cybersecurity in the quantum age: Threats, challenges, and solutions," *International Journal of Advanced Research in Science, Communication and Technology*, pp. 147–151, 11 2023.
- [4] N. Kumar, A. C. Sen, V. Hordichuk, M. Teresa, E. Jaramillo, B. Molodetskyi, D. Amol, and B. Kasture, "Ai in cybersecurity: Threat detection and response with machine learning 1*," 2023.
- [5] J. Manokaran, G. Vairavel, and J. Vijaya, "A novel set theory rule based hybrid feature selection techniques for efficient anomaly detection system in iot edge," in *iQ-CCHESS 2023 - 2023 IEEE International Conference on Quantum Technologies, Communications, Computing, Hardware and Embedded Systems Security*, Institute of Electrical and Electronics Engineers Inc., 2023.
- [6] A. Aksjonov and V. Kyrki, "A safety-critical decision-making and control framework combining machine-learning-based and rule-based algorithms," *SAE International Journal of Vehicle Dynamics, Stability, and NVH*, vol. 7, 2023.
- [7] M. Sarhan, S. Layeghy, M. Gallagher, and M. Portmann, "From zero-shot machine learning to zero-day attack detection," *International Journal of Information Security*, vol. 22, 2023.
- [8] D. Hermawan, N. G. Novianto, and D. Octavianto, "Development of open source-based threat hunting platform," in *2021 2nd International Conference on Artificial Intelligence and Data Sciences (AiDAS)*, pp. 1–6, IEEE, 2021.
- [9] Z. Zhao, C. Xu, and B. Li, "A lstm-based anomaly detection model for log analysis," 2021.
- [10] A. Pranolo, X. Zhou, Y. Mao, B. W. Pratolo, A. P. Wibawa, A. B. P. Utama, A. F. Ba, and A. U. Muhammad, "Exploring lstm-based attention mechanisms with pso and grid search under different normalization techniques for energy demands time series forecasting," *Knowledge Engineering and Data Science*, vol. 7, p. 1, 4 2024.
- [11] Z. Zou, X. Yan, Y. Yuan, Z. You, and L. Chen, "Attention mechanism enhanced lstm networks for latency prediction in deterministic mec networks," *Intelligent Systems with Applications*, vol. 23, 9 2024.
- [12] Q. Kang, E. J. Chen, Z. C. Li, H. B. Luo, and Y. Liu, "Attention-based lstm predictive model for the attitude and position of shield machine in tunneling," *Underground Space (China)*, vol. 13, pp. 335–350, 12 2023.
- [13] H. Studiawan and F. Sohel, "Performance evaluation of anomaly detection in imbalanced system log data," in *2020 Fourth World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4)*, pp. 239–246, IEEE, 2020.
- [14] R. Mohammed, J. Rawashdeh, and M. Abdullah, "Machine learning with oversampling and undersampling techniques: overview study and experimental results," in *2020 11th international conference on information and communication systems (ICICS)*, pp. 243–248, IEEE, 2020.
- [15] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1532–1543, 2014.
- [16] J. Liang, "Confusion matrix: Machine learning," *POGIL Activity Clearinghouse*, 2022.